

# Karzar API - Frontend Integration Guide

Version: API v1

Base path: /api/v1

Content-Type: application/json (except login)

Live contract: /api/openapi.json · Interactive docs: /api/docs

This document answers every contract requirement you originally requested and defines how the storefront frontend should consume the backend.

---

## 0. Quick start checklist

1. Generate TypeScript types from OpenAPI:

```
npx openapi-typescript http://localhost:8000/api/openapi.json -o src/types/api.ts
```

2. Use **Next.js + TypeScript + TanStack Query** (recommended).

3. Build a single API client that:

- Attaches Authorization: Bearer <token> when needed
- Parses the error envelope on non-2xx responses
- Never assumes list endpoints return a raw array

4. Treat `thumbnail: null` as "no image yet" and show a placeholder.

5. Render category menus **recursively** via `subcategories` (unlimited depth).

---

## 1. Core data models (original requirement #1)

### 1.1 Product List View (PLP)

Endpoint: GET /api/v1/products/

Each item in data:

| Field | Type | Notes |
|-------|------|-------|
|-------|------|-------|

|      |          |  |
|------|----------|--|
| `id` | `number` |  |
|------|----------|--|

|       |          |  |
|-------|----------|--|
| `sku` | `string` |  |
|-------|----------|--|

|        |          |  |
|--------|----------|--|
| `name` | `string` |  |
|--------|----------|--|

|             |                   |                                                |
|-------------|-------------------|------------------------------------------------|
| `thumbnail` | `string`   `null` | Primary image URL; `null` if no image uploaded |
|-------------|-------------------|------------------------------------------------|

|              |                   |                                              |
|--------------|-------------------|----------------------------------------------|
| `base_price` | `string`   `null` | Decimal serialized as string, e.g. `"99.99"` |
|--------------|-------------------|----------------------------------------------|

|                |                                 |                     |
|----------------|---------------------------------|---------------------|
| `stock_status` | `"in_stock"`   `"out_of_stock"` | Derived server-side |
|----------------|---------------------------------|---------------------|

|            |                         |  |
|------------|-------------------------|--|
| `category` | `{ id, name }`   `null` |  |
|------------|-------------------------|--|

|         |                         |  |
|---------|-------------------------|--|
| `brand` | `{ id, name }`   `null` |  |
|---------|-------------------------|--|

Example response:

```
{
  "data": [
    {
      "id": 1,
      "sku": "1108-150",
      "name": "Digital Caliper 0-150mm",
      "thumbnail": "https://cdn.example.com/products/1108-150.jpg",
      "base_price": "99.99",
      "stock_status": "in_stock",
      "category": { "id": 1, "name": "Digital Calipers" },
    }
  ]
}
```

```

    "brand": { "id": 1, "name": "Insize" }
  }
],
"meta": {
  "total_count": 5000,
  "skip": 0,
  "limit": 100,
  "has_next": true,
  "has_prev": false
}
}

```

Frontend usage: PLP cards use thumbnail, name, base\_price, stock\_status. Use category / brand for badges/filters. Do not call PDP for each row.

## 1.2 Product Detail View (PDP)

Endpoints:

- `GET /api/v1/products/{product\_id}`
- `GET /api/v1/products/sku/{sku}`

Response shape: ProductDetailResponse

Field | Type | Notes

```

`id`, `sku`, `name` | |
`category_id`, `brand_id` | `number` |
`category`, `brand` | nested brief objects |
`base_price` | `string` \ | `null` |
`stock_quantity` | `string` | Decimal as string
`stock_unit` | `"piece" \ | "kg" \ | "meter" \ | "pack" |
`stock_status` | `"in_stock" \ | "out_of_stock" |
`low_stock` | `boolean` | `true` when quantity < 10
`availability` | `boolean` | `is_active` && stock_quantity > 0`
`thumbnail` | `string` \ | `null` |
`images` | `{ id, url, is_primary }[]` |
`specifications` | object (typed, see below) |
`warranty_text`, `weight_grams`, `pdf_catalog_url` | |
`is_original`, `tax_percent`, `is_active` | |
`created_at`, `updated_at` | ISO 8601 strings |

```

specifications structure (fixed contract):

```

{
  "technical_specs": {
    "range": "0-150mm/0-6\"",
    "accuracy": "+/-0.02mm",
    "resolution": "0.01mm/0.0005\"",
    "material": "Stainless steel",
    "standard": "DIN862",
    "battery_type": "CR2032"
  },
  "features": {
    "waterproof": false,
    "data_output": true,
    "auto_power_off": true,
    "buttons": ["on/off", "zero", "mm/inch"],
    "certification": "Supplied with manufacturer inspection certificate"
  }
}

```

```

},
"dimensions": {
  "L_mm": 236.0,
  "a_mm": 21.0,
  "b_mm": 16.0,
  "c_mm": 16.0,
  "d_mm": 40.0
},
"optional_accessories": [
  "wireless transmitter code 7315-25"
]
}

```

Frontend usage: Build the specs table from specifications.technical\_specs, feature chips from specifications.features, gallery from images (fallback: thumbnail), CTA from availability / stock\_status.

Images: Uploaded by the catalog team via admin (/admin -> Product Images). Until then, thumbnail is null and images is [] - always handle placeholders.

### 1.3 Category tree (mega-menu)

Endpoint: GET /api/v1/categories/tree

Important: Response is not a raw array. It is wrapped:

```

{
  "data": [
    {
      "id": 1,
      "name": "Digital Calipers",
      "parent_id": null,
      "subcategories": [
        {
          "id": 2,
          "name": "Standard Type",
          "parent_id": 1,
          "subcategories": [
            {
              "id": 3,
              "name": "0-150mm Range",
              "parent_id": 2,
              "subcategories": []
            }
          ]
        }
      ]
    }
  ]
}

```

Contract rules:

- Nested key is `**`subcategories`**` (not ``children``)
- `**Unlimited depth**` - render recursively; do not hard-code 2 or 3 levels
- Siblings sorted alphabetically by ``name``
- Filter products by ``category_id`` on PLP (no ``category_slug`` yet)

Frontend usage: Implement a recursive CategoryMenu component that maps subcategories and links to

/products?category\_id={id}.

## 2. Standardized error envelope (original requirement #2)

Every error response uses this shape (never rely on FastAPI's old detail-only format):

```
{
  "error_code": "VALIDATION_FAILED",
  "message": "Request validation failed",
  "details": [
    { "field": "sku", "message": "already exists" }
  ]
}
```

HTTP status | Typical `error\_code`

400 | `BAD\_REQUEST`

401 | `UNAUTHORIZED`

403 | `FORBIDDEN`, `STEP\_UP\_REQUIRED`, `STEP\_UP\_INVALID`, `STEP\_UP\_MISMATCH`

404 | `NOT\_FOUND`

409 | `CONFLICT`

422 | `VALIDATION\_FAILED`

500 | `INTERNAL\_ERROR`

422 validation example:

```
{
  "error_code": "VALIDATION_FAILED",
  "message": "Request validation failed",
  "details": [
    { "field": "query.limit", "message": "Input should be less than or equal to 1000" }
  ]
}
```

Frontend usage: Centralize error parsing. Show message as toast/alert. Map details[] to form field errors when field is present. Branch on error\_code for auth redirects.

## 3. Dynamic filtering on specifications (original requirement #3)

Endpoint: GET /api/v1/products/

### Standard filters (query params)

Param | Type | Description

`skip` | int | Offset (default `0`)

`limit` | int | Page size (default `100`, max `1000`)

`category\_id` | int |

`brand\_id` | int |

`is\_active` | bool |

`search` | string | Name, SKU, brand name

`min\_price`, `max\_price` | decimal |

### Specification filters (JSONB) - two syntaxes

A) JSON filters param (recommended for nested paths):

```
GET /api/v1/products/?filters={"technical_specs.range":"0-150mm"}
GET /api/v1/products/?filters={"features.waterproof":true}
GET /api/v1/products/?filters={"technical_specs.range__icontains":"150"}
```

#### B) Prefixed query params (simple filters):

```
GET /api/v1/products/?spec_brand=insize
GET /api/v1/products/?spec_technical_specs__range=0-150mm
```

Rule: spec\_ prefix; use \_\_ instead of . for nesting.

Merge rule: If both are sent, prefixed params override keys from filters.

Invalid filters JSON -> 400 with error\_code: "VALIDATION\_FAILED".

## 4. Pagination wrapper (original requirement #4)

Rule: List endpoints never return a bare array.

Shape:

```
{
  "data": [ ],
  "meta": {
    "total_count": 5000,
    "skip": 0,
    "limit": 100,
    "has_next": true,
    "has_prev": false
  }
}
```

Currently paginated: GET /api/v1/products/ only.

UI logic:

- Next page: `has\_next === true` -> `skip + limit`
- Previous page: `has\_prev === true` -> `Math.max(0, skip - limit)`
- Page count: `Math.ceil(meta.total\_count / meta.limit)`

Use TanStack Query with queryKey: ["products", { skip, limit, category\_id }].

## 5. Step-up authentication (original requirement #5)

Decision (fixed): secure\_token is returned in the JSON body, sent back via header - not an HttpOnly cookie.

### Flow for destructive admin actions (Delete, Restore)

```
1. POST /api/v1/auth/login      -> access_token (JWT)
2. POST /api/v1/auth/verify-pin -> secure_token (short-lived)
3. DELETE /api/v1/products/{id} -> requires BOTH headers
```

### Step 1 - Login

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded
```

```
username=09123456789&password=yourpassword
```

Note: username = Iranian mobile (09XXXXXXXXXX), not JSON.

Response:

```
{
  "access_token": "eyJ...",
  "token_type": "bearer",
  "expires_in": 1800
}
```

expires\_in is in seconds (ACCESS\_TOKEN\_EXPIRE\_MINUTES x 60).

## Step 2 - Verify PIN (super-admin only)

```
POST /api/v1/auth/verify-pin
Authorization: Bearer <access_token>
Content-Type: application/json
```

```
{ "pin": "your-admin-pin" }
```

Response:

```
{
  "secure_token": "eyJ...",
  "token_type": "step_up",
  "expires_in": 300
}
```

## Step 3 - Destructive request

```
DELETE /api/v1/products/42
Authorization: Bearer <access_token>
X-Step-Up-Token: <secure_token>
```

Without step-up token -> 403:

```
{
  "error_code": "STEP_UP_REQUIRED",
  "message": "Step-up authentication required for this action",
  "details": [{ "field": "X-Step-Up-Token", "message": "Missing step-up token" }]
}
```

Endpoints requiring step-up: DELETE /products/{id}, POST /products/{id}/restore

# 6. Full endpoint map

## Public (storefront)

| Method | Path                        | Response                         |
|--------|-----------------------------|----------------------------------|
| GET    | /api/v1/products/           | ProductListResponse`             |
| GET    | /api/v1/products/{id}       | ProductDetailResponse`           |
| GET    | /api/v1/products/sku/{sku}  | ProductDetailResponse`           |
| GET    | /api/v1/products/{id}/stock | StockStatusResponse`             |
| GET    | /api/v1/categories/tree     | { data: CategoryTreeResponse[] } |
| POST   | /api/v1/auth/register       | UserResponse`                    |

POST | `/api/v1/auth/login` | `Token`

### Admin (Bearer JWT + super\_admin role)

Method | Path | Notes

POST | `/api/v1/products/` | Returns `ProductDetailResponse`

PUT | `/api/v1/products/{id}` | Returns `ProductDetailResponse`

POST | `/api/v1/products/{id}/stock/adjust` | Query: `quantity\_delta`

POST | `/api/v1/auth/verify-pin` | Returns `StepUpTokenResponse`

### Admin + step-up token

Method | Path

DELETE | `/api/v1/products/{id}` | 204 No Content

POST | `/api/v1/products/{id}/restore` | `ProductDetailResponse`

## 7. Recommended frontend architecture

### Stack

Layer | Choice | Why

Framework | **Next.js (App Router)** | SSR/SSG for product SEO

Language | **TypeScript** | Types from OpenAPI

Server state | **TanStack Query** | Cache PLP/PDP, pagination

HTTP | `fetch` wrapper or `ky` | Central auth + error handling

### Suggested folder structure

```
src/
  lib/api/client.ts      # fetch wrapper, auth headers, error parsing
  lib/api/products.ts    # getProducts, getProduct, getProductBySku
  lib/api/categories.ts  # getCategoryTree
  lib/api/auth.ts        # login, verifyPin
  types/api.ts           # generated from OpenAPI
  hooks/useProducts.ts   # TanStack Query hooks
  components/
    product/ProductCard.tsx
    product/ProductGallery.tsx
    product/SpecTable.tsx
    layout/MegaMenu.tsx
```

### Route suggestions (no slug yet)

Page | Route | Data source

PLP | `/products?category\_id=1&skip=0&limit=24` | `GET /products/`

PDP | `/products/{id}` or `/products/sku/{sku}` | `GET /products/{id}` or `/sku/{sku}`

Mega-menu | layout | `GET /categories/tree` -> `data`

URLs use numeric id / sku for now. Slug-based SEO URLs are not in the API yet.

## 8. Known limitations (plan UI accordingly)

Topic | Status | Frontend action  
 `thumbnail` | `null` until images uploaded in admin | Placeholder image  
 `slug` | Not in API | Use `/products/[id]` or `/products/sku/[sku]`  
 Category filter | `category\_id` (int), not slug | Read `id` from tree nodes  
 Image upload API | Admin panel only | Not a storefront concern  
 Open `/register` | Anyone can register | Storefront register OK

---

## 9. Contract summary - 5 requirements

# | Requirement | Status | How to use  
 1 | PLP minimal fields | Done | `GET /products/` -> `data[]`  
 1 | PDP + specifications | Done | `GET /products/{id}` -> `specifications`  
 1 | Category tree | Done | `GET /categories/tree` -> `data[]`, recursive `subcategories`  
 2 | Error envelope | Done | Parse `error\_code`, `message`, `details[]`  
 3 | Spec filtering | Done | `filters` JSON and/or `spec\_\*` params  
 4 | Pagination wrapper | Done | `data` + `meta`; never expect raw array  
 5 | Step-up PIN | Done | `verify-pin` -> `X-Step-Up-Token` header

---

## 10. Environment variables (frontend)

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

All API calls: `\${NEXT\_PUBLIC\_API\_URL}/api/v1/...

---

Source of truth: If this document and the running server disagree, trust `/api/openapi.json`.

\*Karzar Industrial Lathe Tools API - Frontend Integration Guide\*